

Lab 02 – Classes, Properties, Enums

Title: Object-Oriented Programming – Classes and Properties

Objectives

The goal here was to move beyond simple procedural programming and start designing a proper object-oriented model for our application. I needed to create the core entity – the Angajat (Employee) class – that would represent all the data we need to store about an employee. This lab focused on understanding different types of properties: auto-implemented properties for simple data storage, and computed properties for values that are derived from other data.

Implementation

I created a new Class Library project called SalariiModel. This keeps the business logic separate from the UI, which is a good practice for maintainability.

Enums

```
// Departament.cs
namespace SalariiModel
{
    public enum Departament
    {
        IT,
        Marketing,
        Finante,
        HR,
        Productie,
        Logistica
    }
}
```

```

    }
}

// TipSalariu.cs

namespace SalariiModel
{
    public enum TipSalariu
    {
        Lunar,
        Orar
    }
}

```

Angajat Class

```

using System;

namespace SalariiModel
{
    public class Angajat
    {
        // Auto-implemented properties - simple data storage
        public int Id { get; set; }
        public string Nume { get; set; }
        public string Prenume { get; set; }
        public string Functie { get; set; }
        public Departament Departament { get; set; }
        public TipSalariu TipSalariu { get; set; }
        public double SalariuBrut { get; set; }
        public double TarifOra { get; set; }
        public int OreLucreate { get; set; }
    }
}

```

```

public double Bonusuri { get; set; }

public DateTime DataAngajarii { get; set; }

// Computed property - concatenates Nume and Prenume
public string NumeCompleat => $"{Nume}{Prenume}";

// Computed property with conditional logic
public double SalariuBrutEfectiv
{
    get
    {
        if (TipSalariu == TipSalariu.Orar)
            return TarifOra * OreLucreate + Bonusuri;
        else
            return SalariuBrut + Bonusuri;
    }
}

// Computed property with constant deduction rate
private const double PROCENT_IMPOZIT = 0.25; // 25% total deductions
public double SalariuNet => SalariuBrutEfectiv * (1 - PROCENT_IMPOZIT);

// Constructors
public Angajat()
{
    DataAngajarii = DateTime.Today;
}

public Angajat(string nume, string prenume, string functie, Departament departament,
    TipSalariu tipSalariu, double salariuBrut, double tarifOra, int oreLucreate,

```

```

        double bonusuri, DateTime dataAngajarii)
    {
        Nume = nume;
        Prenume = prenume;
        Functie = functie;
        Departament = departament;
        TipSalariu = tipSalariu;
        SalariuBrut = salariuBrut;
        TarifOra = tarifOra;
        OreLucrate = oreLucrate;
        Bonusuri = bonusuri;
        DataAngajarii = dataAngajarii;
    }

    // Returns a formatted string with all employee details
    public string Info()
    {
        return $"ID: {Id}, {NumeComplet}, Funcție: {Functie}, " +
            $"Departament: {Departament}, Salariu Net: {SalariuNet:F2} RON";
    }
}

```

Why I Used This Approach

Auto-Implemented Properties: These are the simplest form of properties in C#. The compiler automatically generates a private backing field. This reduces boilerplate code while maintaining encapsulation.

Enums: Using enums for `Departament` and `TipSalariu` restricts values to a predefined set, preventing errors from typos or invalid entries. This makes the code more self-documenting and type-safe.

Computed Properties: SalariuBrutEfectiv and SalariuNet are calculated based on other properties. This ensures consistency – if any underlying data changes, the computed values update automatically. The business logic is encapsulated within the class where it belongs.

Constant for Tax Rate: Using const for PROCENT_IMPOZIT makes it clear that this value should not change at runtime. If the tax rate changes, I only need to update one place.

Constructor Overloading: Providing both a parameterless constructor and a parameterized constructor gives flexibility – I can create empty objects or fully initialized ones as needed.

Testing & Results

I tested the class by creating instances with different combinations of properties:

```
// Test for hourly employee
```

```
var angajatOrar = new Angajat
```

```
{
```

```
    Nume = "Popescu",
```

```
    Prenume = "Ion",
```

```
    Functie = "Programator",
```

```
    Departament = Departament.IT,
```

```
    TipSalariu = TipSalariu.Orar,
```

```
    TarifOra = 50,
```

```
    OreLucrate = 160,
```

```
    Bonusuri = 200
```

```
};
```

```
// SalariuBrutEfectiv = 50 * 160 + 200 = 8200
```

```
// SalariuNet = 8200 * 0.75 = 6150
```

```
// Test for salaried employee
```

```
var angajatLunar = new Angajat
```

```
{
```

```
    Nume = "Ionescu",
```

```
    Prenume = "Maria",
```

```
    Functie = "Manager",  
    Departament = Departament.Marketing,  
    TipSalariu = TipSalariu.Lunar,  
    SalariuBrut = 8000,  
    Bonusuri = 200  
};  
  
// SalariuBrutEfectiv = 8000 + 200 = 8200  
  
// SalariuNet = 8200 * 0.75 = 6150
```

Both employees correctly calculated the same net salary of 6150 RON despite having different pay structures.

Reflection

This lab helped me understand the difference between storing data and computing data. Computed properties are powerful because they ensure consistency – if the tax rate changes, I only need to update one place. I also appreciated how enums make the code more self-documenting and type-safe. The separation of the model into its own project (Class Library) will pay dividends as the application grows.